

COMP1531

Software Engineering Fundamentals

Week 7

Persistence

In this Lecture:

— — —

- Define **data persistence** and explain why it matters in modern systems.
- Identify where persistence fits within a **software architecture's data layer**.
- Describe different **data storage mechanisms** (in-memory, in-file, in-database).
- Implement **basic persistence** using JSON file storage.

Why Data Persistence?

— — —

- Data is a **critical part of almost every software system**.
- When a program stops, data in memory disappears.
- Persistence allows **information to be retained** even after the program stops running.
- Ensures **consistency and reliability** across multiple sessions or users.
- Forms the **foundation for databases, backups, and data analytics**.
- Persistence is essential for **real-world applications**

Data

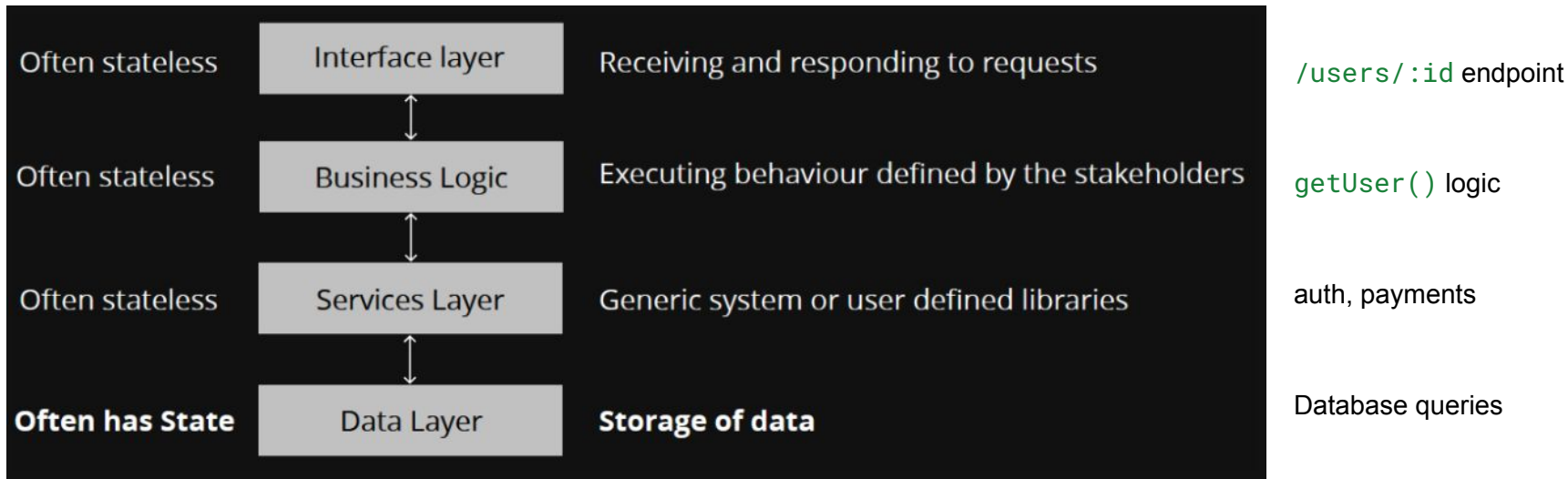
— — —

- Data is also critical to allow an application to function - as it allows for the storage of state.
- Data (and "big data") are becoming huge topics in the world of computing and mathematics. This course does not address these topics.

Data In Applications

- Data is part of every software application (eg. variables)
- However, when we are referring to data in a software, we are referring to the “data layer” - a part of the software that is focussed solely on storing data and maintaining a state for longer term
- Let's take a close look at the data layer.

Eg:



Data Layers ("Databases")

— — —

Database: A word used to describe a store of data. Databases are typically what a data layer consists of.

There are 3 main ways to store data:

1. In-memory (non-persistent)
2. In-file (filesystem direct storage)
3. In-database (Relational SQL, NoSQL)

As you move down the list, barrier to entry becomes higher, but so does performance.

To learn more about relational SQL, study COMP3311.

Data Layers ("Databases")

— — —

But don't all applications store data? E.g. Variables? Memory? Registers?
Something?

Yes! But we're talking about a much longer type of data storage...

Storing Data: Persistence

— — —

- Persistence: When program state outlives the process that created it. This is achieved by storing the state as data in computer data storage.
- Persistence can be interpreted very broadly, though in this course we will define it as storing data to disk.



Storing Data: Persistence

— — —

Can we modify our project server to persistently store data?

How would we do that?

Adding Persistence To The Project

— — —

- For the project, we require you as part of iteration 2 to add a persistence layer.
- Rather than ask you to add something complicated like an SQL database, we're simply asking you to use a simple in-file database.
- All this needs to consist of is loading and storing the state of your program in a JSON string that is stored on file.

Adding Persistence To The Project

— — —

To do this, we can simply use our code samples from an earlier week lecture on JSON.

But when do you load & save this data?

- Load: On server start, open the JSON file and set the global variable to be the parsed JSON object
- Save: Either based on a timer, or at the end of every route call, JSON stringify your global variable and store in a file

Key Takeaways

— — —

- Persistence keeps data alive beyond runtime.
- Simple file-based persistence = first step toward real databases.
- It ensures
 - Reliability - because data survives beyond the running process.
 - Recovery - restoring the system to a correct (last known) state after a failure
 - Scalability - can handle **more users, data, or requests** by adding more resources and multiple instances to coordinate over a shared, stable data layer.

Feedback

COMP1531 Feedback Survey

